دالة main() في C++

1 مُحددات اللغة — Constraints

الحالة	التصنيف	التفصيل
`int main()`	✅ **إلزامي**	النوع الوحيد المسموح به في معيار C++
`return(0);`	✅ **إلزامي** (ممارسة صحيحة)	لإنهاء البرنامج وإخبار نظام التشغيل بالنجاح
كتابة `int` قبل `main`	🟡 **اختياري**	لأن النوع الافتراضي للدوال هو `int` أصلاً
`void main()`	❌ **ممنوع**	مخالف صريح لمعيار C++، يُسبب Undefined Behavior
استدعاء `main()` من دالة أخرى	❌ **ممنوع**	معيار C++ يحظر صراحةً استدعاء `main()` داخل البرنامج

2 صراع المعايير — C vs C++ Standards

[القاعدة الصارمة]: C++ أكثر صرامة من ANSI C في موضوع `main()` بشكل كامل

عنصر المقارنة	ANSI C	C++
نوع الإرجاع	غير إلزامي (مقبول بدونه)	`int` إلزامي بنص المعيار
`void main()`	مقبول في بعض التطبيقات	❌ محظور — يُسبب Error في GCC
حذف `return`	سلوك غير محدد	المُصرّف يُضيفها تلقائياً (`return 0;`)
الشكل القياسي	`main() { }`	`int main() { return(0); }`
مستوى التشدد	مرن (Loosely Typed)	صارم (Strongly Typed)

3 فخاخ المجمِّع — Compiler Traps

[منطقة الخطر/الفخ #1 — Compile-time Error]:

كتابة `void main()` في بيئة GCC الحديثة تُعطي خطأ فورياً:

```text
ERROR: ::main must return int
```

[منطقة الخطر/الفخ #2 — Warning فقط، لا Error]:

حذف `return(0);` من نهاية `main()` لا يُوقف التصميم — المُصرّف يُضيف `return 0;` تلقائياً، لكنه يُصدر Warning:

```text
warning: control reaches end of non-void function
```

[منطقة الخطر/الفخ #3 — Runtime Undefined Behavior]:

كتابة `void main()` في MSVC (Visual Studio) قد تُصرَّف بدون خطأ، لكن سلوك البرنامج عند الإنهاء **يكون غير محدد** (Undefined)

نوع الخطأ	السبب	النتيجة
Compile-time Error	`void main()` في GCC	رفض الترجمة كلياً
Warning	حذف `return(0);`	البرنامج يعمل لكن بتحذير
Undefined Behavior	`void main()` في MSVC	يعمل بنتائج غير متوقعة

4 منطق التتبع — Execution Flow

[القاعدة الصارمة]: ترتيب تنفيذ البرنامج هو:

```text
نظام التشغيل → return(0) → تنفيذ العبارات → main() نظام التشغيل → تحميل البرنامج → استدعاء
```

- القيمة `0` في `return(0)` تعني **نجاح البرنامج**
- القيمة `1` أو أي رقم غير صفري تعني **فشل البرنامج** وتُبلَّغ لنظام التشغيل
- `main()` هي نقطة البداية **الوحيدة** — لا يمكن لأي دالة أخرى أن تبدأ التنفيذ

---

5 قسم "ماذا لو؟" — Edge Cases

السيناريو	ماذا يحدث؟
`void main()` في GCC	Compile-time Error — رفض تام
`void main()` في MSVC	يُصرَّف مع Warning — سلوك غير محدد
حذف `return(0);`	المُصرّف يُضيفها تلقائياً فقط في `main()`
`return(1);` بدلاً من `return(0);`	البرنامج يُخبر النظام بأنه **فشل**
استدعاء `main()` من دالة أخرى	محظور معيارياً في C++ — Undefined Behavior
حذف `int` وكتابة `main()` فقط	Warning في C99+، لأن `int` افتراضي

🔴 سؤال (1) — What is the output?

```cpp
void main()
{
    cout << "Iraq";
}
```

[سؤال امتحاني متوقع]:

> ما نتيجة تصريف هذا الكود في بيئة GCC؟
>
> - أ) يطبع: `Iraq`
> - ب) ✅ يُعطي Compile-time Error
> - ج) يطبع ثم يُعطي Warning
> - د) لا شيء — البرنامج يعمل بصمت

الإجابة: ب — لأن `void main()` محظور في C++ المعياري

🟡 سؤال (2) — What is the output?

```cpp
int main()
{
    cout << "Hello";
}
```

[سؤال امتحاني متوقع]:

> ماذا يحدث عند تصريف هذا الكود (بدون `return`)؟
>
> - أ) Compile-time Error
> - ب) ✅ يطبع `Hello` مع Warning
> - ج) يطبع `Hello` بدون أي مشكلة
> - د) Undefined Behavior دائماً

الإجابة: ب — المُصرّف يُضيف `return 0;` تلقائياً لكن يُصدر تحذيراً

الدوال المضمنة (Inline) والوسائط الافتراضية(Default Arguments)

🔷 القسم الأول — الدوال المضمنة (Inline Functions)

القاعدة الذهبية

[القاعدة الذهبية #1]: الكلمة المفتاحية `inline` هي **طلب** (Request) للمُصرّف وليست **أمراً** (Command) — للمُصرّف الحق الكامل في تجاهلها

[القاعدة الذهبية #2]: الهدف من `inline` هو **إزالة تكلفة الاستدعاء** (Overhead) عن طريق استبدال سطر الاستدعاء بكود الدالة الفعلي مباشرةً

```cpp
// المُصرّف يحوّل هذا الاستدعاء:
result = square(5);

// إلى main: هذا الكود مباشرةً داخل
result = 5 * 5;
```

جدول الحالات القانونية وغير القانونية

[جدول الفروقات — متى يتجاهل المُصرّف كلمة `inline` ؟]

الحالة	تعمل `inline` ؟	السبب
دالة بسيطة (تعبير واحد)	✅ نعم	مثالية للتوسيع
دالة تحتوي على `Loop`	❌ يتجاهلها	التوسيع قد يُضاعف الكود
دالة تحتوي على `switch`	❌ يتجاهلها	تعقيد التدفق
دالة تحتوي على `goto`	❌ يتجاهلها	تدمير ترتيب التنفيذ
دالة `void` تحتوي على `return`	❌ يتجاهلها	كسر تدفق التوسيع
دالة تحتوي على `static variable`	❌ يتجاهلها	المتغير الثابت يحتاج عنوان ثابت في الذاكرة
دالة **استدعاء ذاتي** (Recursive)	❌ يتجاهلها	سيُسبب توسيعاً لا نهائياً

فخ الـ Inline

[منطقة الخطر #1]: الطالب يظن أن كتابة `inline` **تضمن** التوسيع — والصواب أنه **لا ضمان** على الإطلاق

[منطقة الخطر #2]: الطالب يظن أن الدالة التكرارية `inline` ستُعطي Error — والصواب أن المُصرّف **يتجاهلها بصمت** ويُصرّفها كدالة عادية بدون أي خطأ

بنك Tracing — Inline Functions

[تمرين Tracing #1] — What is the output?

```cpp
inline int add(int a, int b)
{
    return a + b;
}
int main()
{
    cout << add(3, 4);
    return 0;
}
```

> ✅ المخرج: `7` — الدالة بسيطة بدون loops، سيقبل المُصرّف الطلب

[تمرين Tracing #2] — لماذا لا يعمل هذا كـ inline؟

```cpp
inline int factorial(int n)
{
    if(n == 0) return 1;
    return n * factorial(n-1);
}
```

> ⚠️ السبب: دالة تكرارية — المُصرّف يتجاهل كلمة `inline` ويُصرّفها كدالة عادية

[سؤال امتحاني متوقع #1]:

> أي من الدوال التالية لا يُعمل فيها التوسيع المضمن؟
>
> - أ) دالة تُرجع مجموع عددين
> - ب) ✅ دالة تحتوي على `for loop`
> - ج) دالة تُرجع قيمة ثابتة
> - د) دالة تُرجع ناتج ضرب عددين

🔶 القسم الثاني — الوسائط الافتراضية (Default Arguments)

قاعدة اليمين إلى اليسار — Trailing Arguments

[القاعدة الذهبية #3]: الوسائط الافتراضية **يجب** أن تكون دائماً في **نهاية قائمة المعاملات (من اليمين)**. لا يمكن وضع وسيط افتراضي قبل وسيط غير افتراضي

[جدول الحالات القانونية وغير القانونية — Default Arguments]

التعريف	الحالة	السبب
`void f(int a, int b=5, int c=10);`	✅ **قانوني**	الافتراضيان في اليمين
`void f(int a=1, int b, int c=10);`	❌ **غير قانوني**	`b` بعد `a` الافتراضي ليس افتراضياً
`void f(int a=1, int b=2, int c=3);`	✅ **قانوني**	الكل افتراضي من اليمين
`void f(int a=0, int b);`	❌ Compile Error	الافتراضي قبل غير الافتراضي

مكان التعريف — Declaration vs Definition

[القاعدة الذهبية #4]: القيمة الافتراضية تُوضع في الـ **Prototype (Declaration)** فقط — إذا أُعيد ذكرها في الـ Definition سيُعطي المُصرّف **Compile-time Error**

```cpp
// الملف .h — الطريقة الصحيحة
void display(int x, int y = 10); // ✅ القيمة هنا فقط

// الملف .cpp — الطريقة الصحيحة
void display(int x, int y)       // ✅ بدون تكرار القيمة
{
    cout << x << " " << y;
}
```

فخ الـ Ambiguity مع Overloading

[فخ الـ Ambiguity — الأخطر في التنافسي]: عندما يجتمع Default Argument مع Function Overloading، قد يعجز المُصرّف عن اختيار الدالة الصحيحة فيُعطي **Ambiguity Error**

```cpp
void show(int a = 10);          // دالة 1
void show(int a, int b = 20); // دالة 2

int main()
{
    show(5);      // ❌ AMBIGUITY! — هل هي الدالة 1 أم 2؟
    show(5, 10); // ✅ واضحة — الدالة 2
    show();      // ✅ واضحة — الدالة 1
}
```

[منطقة الخطر — Overloading Conflict]

نمط الاستدعاء	النتيجة
`show()`	✅ يستدعي الدالة 1 فقط (لا غموض)
`show(5)`	❌ **Ambiguity Error** — تنطبق على كلتيهما
`show(5, 10)`	✅ يستدعي الدالة 2 فقط (لا غموض)

منطق الاستدعاء — Execution Flow

[القاعدة الذهبية #5]: عند تمرير وسيط واحد لدالة ذات 3 معاملات (اثنان افتراضيان)، يُسند الوسيط المُمرَّر **للأول من اليسار**، والافتراضيان للثاني والثالث

```cpp
void func(int a, int b=20, int c=30);

func(5);        // a=5,  b=20, c=30 ✅
func(5, 10);    // a=5,  b=10, c=30 ✅
func(5, 10, 15);// a=5,  b=10, c=15 ✅
func(5, , 15);  // ❌ Syntax Error — لا يمكن تخطي وسيط وسطي
```

بنك Tracing — Default Arguments

[تمرين Tracing #3] — What is the output?

```cpp
void print(int x, int y=5, int z=10)
{
    cout << x+y+z;
}
int main()
{
    print(2, 3);
    return 0;
}
```

> ✅ المخرج: `15` — لأن `x=2, y=3, z=10` (z أخذ قيمته الافتراضية)

[سؤال امتحاني متوقع #2 — الأكثر تكراراً في العراق]:

> أي من التعريفات التالية يُعطي Compile-time Error؟
>
> - أ) `void f(int a, int b=5, int c=10);`
> - ب) `void f(int a=1, int b=2, int c=3);`
> - ج) ✅ `void f(int a=1, int b, int c=10);`
> - د) `void f(int a, int b, int c=10);`

الوسائط الافتراضية (Default Arguments)

⚡ القاعدة الأولى — منطق اليمين إلى اليسار

[القاعدة الصارمة #1 — The Right-to-Left Rule]:

يجب أن تكون الوسائط الافتراضية دائماً في **نهاية قائمة المعاملات** وتُبنى من اليمين إلى اليسار. إذا أُعطي معامل قيمة افتراضية، فيجب أن تمتلك **كل المعاملات التي تليه** على اليمين قيماً افتراضية أيضاً

لماذا هذه القاعدة منطقية؟ لأن المُصرّف يُوزّع القيم المُمرّرة من **اليسار إلى اليمين** بالترتيب — فإذا تُرك فراغ في الوسط لا يعرف المُصرّف أي معامل تركته

```cpp
// ✅ صحيح — الافتراضي من اليمين
float amount(float p, int n, float rate = 0.15);

// ❌ خطأ — الافتراضي في الوسط
float amount(float p = 5000, int n, float rate);
```

⚡ القاعدة الثانية — منطق المُصرّف في الاستدعاء

[القاعدة الصارمة #2]:

المُصرّف يملأ المعاملات من **اليسار إلى اليمين** بالقيم المُمرّرة، ثم يستخدم القيم الافتراضية **للمعاملات المتبقية على اليمين**

```cpp
float amount(float principal, int period, float rate = 0.15);

// استدعاء 1: تمرير قيمتين فقط
amount(5000, 7);
// → principal=5000, period=7, rate=0.15 (افتراضي) ✅

// استدعاء 2: تمرير جميع القيم
amount(5000, 7, 0.12);
// → principal=5000, period=7, rate=0.12 (يُلغي الافتراضي) ✅

// استدعاء 3: محاولة تخطي وسيط وسطي
amount(5000, , 0.12);
// → ❌ Syntax Error — لا يمكن تخطي وسيط وسطي
```

⚡ القاعدة الثالثة — مكان التعريف

[القاعدة الصارمة #3 — Declaration vs. Definition]:

القيم الافتراضية **تُوضع في الـ Prototype (Declaration) فقط**. إذا كان الـ Declaration والـ Definition منفصلَيْن، فلا يجوز تكرار القيمة الافتراضية في الـ Definition — سيُعطي **Compile-time Error فوري**

```cpp
// ✅ الموضع الصحيح — header.h الملف
float amount(float p, int n, float rate = 0.15);

// ✅ بدون تكرار القيمة — source.cpp الملف
float amount(float p, int n, float rate)
{
    return p * n * rate; // تعريف بدون قيم افتراضية
}

// ❌ أيضاً Definition خطأ — تكرار القيمة في الـ
float amount(float p, int n, float rate = 0.15) // ERROR!
{
    return p * n * rate;
}
```

> [منطقة الخطأ]: الطالب يكتب القيمة الافتراضية في كلا المكانيْن (Prototype + Definition) ظناً أن ذلك "تأكيد". الصواب: في الـ Prototype فقط

📊 جدول الصواب والخطأ — Legal vs Illegal

التعريف	الحالة	السبب
`void f(int a, int b=5, int c=10);`	✅ Legal	الافتراضيان في اليمين بالترتيب
`void f(int a=1, int b=2, int c=3);`	✅ Legal	الكل افتراضي من اليمين
`void f(int a, int b, int c=10);`	✅ Legal	آخر معامل فقط افتراضي
`void f(int a=1, int b, int c=10);`	❌ Illegal	`b` بعد `a` الافتراضي ليس افتراضياً
`void f(float p=5000, int n, float r);`	❌ Illegal	الافتراضي في البداية، الباقيان ليسا كذلك
`void f(int a=0, int b=0, int c);`	❌ Illegal	`c` على اليمين بدون قيمة افتراضية
`float f(float p, int n, float r=0.15);` + `float f(float p, int n, float r=0.15){...}`	❌ Illegal	تكرار القيمة في Prototype و Definition

🎯 مزايا MCQ — خوارزميات الحل السريع

استناداً لنقاط 5 و6 من المنهج، هذه الجمل تصلح كـ**فلاتر إجابة** مباشرة في الامتحان:

- **[خوارزمية 1]:** إذا رأيت قيمة افتراضية **في وسط** قائمة المعاملات قبل معامل غير افتراضي ← **الإجابة فوراً:** Compile Error
- **[خوارزمية 2]:** إذا رأيت القيمة الافتراضية في جسم الدالة (Definition) مع وجود Prototype ← **الإجابة:** Compile Error
- **[خوارزمية 3]:** الهدف من Default Arguments هو **دمج دوال متشابهة في دالة واحدة** وتقليل تكرار الكود — إذا سألك السؤال عن "ميزة الدمج" ← **الإجابة:** Default Arguments
- **[خوارزمية 4]:** إذا مُرِّرت قيمة صريحة، فهي **تُلغي** (Overrides) القيمة الافتراضية تماماً

🔍 قسم تتبع المخرج — Tracing Exercise

كود: حساب الفائدة البنكية

```cpp
#include<iostream>
using namespace std;

// Prototype: القيمة الافتراضية هنا فقط
float amount(float principal, int period, float rate = 0.15);

float amount(float principal, int period, float rate)
{
    return principal * period * rate;
}

int main()
{
    // استدعاء 1: تمرير جميع القيم
    cout << amount(5000, 3, 0.10) << endl;

    // استدعاء 2: قيمة واحدة افتراضية ناقصة
    cout << amount(5000, 3) << endl;

    // استدعاء 3: محاولة حذف وسيط من الوسط
    cout << amount(5000) << endl;

    return 0;
}
```

[سؤال امتحاني متوقع — Tracing]:

> ما مخرجات الاستدعاءات الثلاثة؟

الاستدعاء	القيم الفعلية	الحساب	المخرج
`amount(5000, 3, 0.10)`	`p=5000, n=3, rate=0.10`	`0.10×3×5000`	✅ 1500
`amount(5000, 3)`	`p=5000, n=3, rate=0.15`	`0.15×3×5000`	✅ 2250
`amount(5000)`	`period` مجهول	لا حساب	❌ Compile Error

> [منطقة الخطأ]: الاستدعاء الثالث يُعطي Compile Error وليس صفراً أو قيمة افتراضية — لأن `period` لا يمتلك قيمة افتراضية وهو معامل إلزامي

🏆 سؤال MCQ متوقع — من التنافسي

[سؤال امتحاني متوقع #1]:

> أيٌّ من التعريفات التالية يُسبب Compile Error؟
>
> - أ) `void calc(int x, int y=0, int z=0);`
> - ب) `void calc(int x, int y, int z=10);`
> - ج) ✅ `void calc(int x=5, int y, int z=10);`
> - د) `void calc(int x=1, int y=2, int z=3);`

[سؤال امتحاني متوقع #2]:

> في دالة `void f(int a, int b=2, int c=3)`، عند الاستدعاء `f(10, 5)`، ما قيمة `c`؟
>
> - أ) 0
> - ب) 5
> - ج) ✅ 3 (القيمة الافتراضية)
> - د) Compile Error

الوسائط الثابتة (Const Arguments)

1 الضمانة الأمنية — The Security Contract

[القاعدة الصارمة #1 — عقد الحماية]:

كلمة `const` أمام معامل الدالة هي **عقد رسمي** بين المبرمج والمُصرِّف مفاده: "أنا أعدك أن هذه الدالة لن تُعدِّل هذه القيمة أبداً". إذا نقضت هذا العقد داخل الدالة، سيعاقبك المُصرِّف بـ **Compile-time Error** فوري — قبل أن يُشغِّل البرنامج أي سطر كود.

```cpp
// العقد: الدالة تَعِد بعدم تعديل p
int strlen(const char *p);

// العقد: الدالة تَعِد بعدم تعديل s
int length(const string &s);
```

2 نوع الخطأ — Compile-time Error (Static Analysis)

[القاعدة الصارمة #2]:

خطأ `const` لا يظهر عند تشغيل البرنامج (Runtime) — بل يكتشفه المُصرِّف في مرحلة التجميع (Compile-time) قبل إنتاج أي ملف تنفيذي. هذا يعني أن برنامجك **لن يُصرَّف أصلاً** إذا انتهكت قيد `const`.

مرحلة الخطأ	`const` Violation	خطأ عادي
متى يظهر؟	Compile-time (قبل التشغيل)	قد يظهر Runtime
هل يعمل البرنامج؟	❌ لا يُصرَّف أبداً	✅ يعمل ثم يفشل
نوع التحليل	Static Analysis	Dynamic Analysis

3 الشرط الجوهري — متى تكون `const` ذات قيمة حقيقية؟

هذه النقطة هي **لب الامتحان التنافسي**.

[القاعدة الصارمة #3 — التمرير بالقيمة vs المرجع والمؤشر]:

عند **التمرير بالقيمة (Pass by Value)**، تعمل الدالة على **نسخة مؤقتة** من البيانات — أي أن تعديلها لا يؤثر على الأصل أصلاً. لذا `const` هنا **عديمة الفائدة الفعلية** (لا تحمي شيئاً حقيقياً).

أما عند **التمرير بالمرجع أو المؤشر**، تعمل الدالة مباشرة على **البيانات الأصلية في الذاكرة** — وهنا `const` تصبح **درع الحماية الوحيد** من التعديل غير المقصود.

4 منطق الـ Pointer — قبل النجمة أم بعدها؟

[القاعدة الصارمة #4 — موضع `const` مع المؤشرات]:

هذا الفرق من أكثر ما يُضلِّل الطلاب في التنافسي:

```cpp
// 1 النمط: const قبل النجمة
const char *p;
// *p) لا يمكن تغيير) الثابت هو: القيمة المُشار إليها
// (يمكن أن يشير لمكان آخر p) لكن يمكن تغيير العنوان نفسه

// 2 النمط: const بعد النجمة
char * const p;
// (لا يمكن أن يشير لمكان آخر p) الثابت هو: العنوان
// (يمكن تغييرها *p) لكن يمكن تغيير القيمة المُشار إليها

// 3 النمط: const في المكانين
const char * const p;
// لا يمكن تغيير العنوان ولا القيمة
```

[جدول المقارنة — مواضع `const` مع المؤشر]:

الصيغة	الثابت	المتغير	الاستخدام الشائع
`const char *p`	القيمة `p*`	العنوان `p`	حماية البيانات المقروءة كـ `strlen`
`char * const p`	العنوان `p`	القيمة `p*`	مؤشر ثابت يشير لمكان محدد
`const char * const p`	كلاهما	لا شيء	حماية مطلقة

5 منطقة الفخ — The Trap

[منطقة الفخ #1 — محاولة التعديل داخل الدالة]:

الطالب يظن أن `const` مجرد "تحذير" وأن الكود سيعمل — الصواب أن المُصرّف **يرفض التجميع كلياً**:

```cpp
int strlen(const char *p)
{
    *p = 'A'; // ❌ Compile Error!
    // رسالة المُصرّف:
    // "assignment of read-only location '*p'"
    p++;      // ✅ (تغيير العنوان وليس القيمة) مسموح
}
```

[منطقة الفخ #2]:

الطالب يخلط بين حماية **العنوان** وحماية **القيمة** في المؤشرات — دائماً اسأل نفسك: "النجمة على يمين *const* أم يسارها؟"

---

6 تمرين Tracing — حدد سطر الخطأ

```cpp
#include<iostream>
using namespace std;

void process(const int &x, const char *msg)
{
    cout << x << endl;      // سطر 1
    x = 100;                // سطر 2
    cout << msg << endl;    // سطر 3
    *msg = 'Z';             // سطر 4
    msg++;                  // سطر 5
}

int main()
{
    int val = 50;
    process(val, "Hello");
    return 0;
}
```

[سؤال امتحاني متوقع — Tracing]:

> حدد السطر (أو الأسطر) التي تُسبب Compile Error:

السطر	الكود	النتيجة		السبب
سطر 1	`cout << x`	✅ لا خطأ		قراءة فقط — مسموح
سطر 2	`x = 100`	❌ Compile Error		`x` محمي بـ `const &` — لا يُعدَّل
سطر 3	`cout << msg`	✅ لا خطأ		قراءة فقط — مسموح
سطر 4	`*msg = 'Z'`	❌ Compile Error		القيمة المُشار إليها محمية بـ `const char *`
سطر 5	`++msg`	✅ لا خطأ		تغيير **العنوان** مسموح — `const` قبل النجمة لا بعدها

🏆 أسئلة MCQ متوقعة

[سؤال #1]:

> متى تكون `const` ذات أهمية فعلية مع معاملات الدوال؟
>
> - أ) عند التمرير بالقيمة دائماً
> - ب) عند التمرير بالمرجع أو المؤشر فقط ✅
> - ج) مع جميع طرق التمرير بنفس الأهمية
> - د) فقط مع المتغيرات العالمية

[سؤال #2]:

> في `int strlen(const char *p)`، أيٌّ من العمليتين التاليتين غير مسموح بها؟
>
> - أ) `p++` (تحريك المؤشر للأمام)
> - ب) `p = p + 2` (تغيير العنوان)
> - ج) ✅ `*p = 'X'` (تعديل القيمة المُشار إليها)
> - د) `char c = *p` (قراءة القيمة)

لدوال الزائدة(Function Overloading)

1 الضمانة الأمنية — عقد الـ Compiler

[القاعدة الصارمة #1 — عقد التمييز]:

الـ Compiler لا يُميّز بين الدوال الزائدة عن طريق **اسم الدالة** (فهي متطابقة)، ولا عن طريق **نوع الإرجاع** — العقد الوحيد الذي يعتمد عليه هو **عدد الوسائط + نوعها فقط**

```cpp
// ✅ Overloading صحيح — يختلفان في النوع
int square(int x);
float square(float x);

// ❌ NOT Overloading — نفس المعاملات، نوع إرجاع مختلف فقط
int sum(int a, int b);
float sum(int a, int b);   // Compile Error! لا يُعدّ Overloading
```

2 خطوات اختيار الدالة — منطق الـ Compiler

[القاعدة الصارمة #2 — تسلسل ثلاثي لا يتغير]

يتبع المُصرّف هذا التسلسل بالترتيب الصارم:

```text
1 الخطوة ──► Exact Match (تطابق تام)
     │ فشل؟
     ▼
2 الخطوة ──► Integral Promotion (ترقية ضمنية)
     │        char→int  /  float→double
     │ فشل؟
     ▼
3 الخطوة ──► Built-in Conversion (تحويل مدمج)
     │        int→long / int→double
     │ أكثر من تطابق؟
     ▼
   AMBIGUITY ERROR ❌
```

[جدول المقارنة — الخطوات الثلاث]:

الخطوة	الاسم	مثال	النتيجة
1	Exact Match	تمرير `int` لدالة تأخذ `int`	✅ تُنفَّذ فوراً
2	Promotion	تمرير `char` لدالة تأخذ `int`	✅ ترقية آمنة
3	Built-in Conversion	تمرير `int` لدالتَيْ `long` و `double`	❌ Ambiguity Error

---

3 الفخ الأخطر — مشكلة الغموض (Ambiguity)

[منطقة الفخ #1 — فخ التحويل المزدوج]:

الطالب يظن أن المُصرِّف سيختار "أقرب نوع" — لكن إذا أمكن التحويل لأكثر من دالة واحدة في الخطوة 3، فالنتيجة **Compile Error** وليس اختياراً عشوائياً

```cpp
long square(long n);
double square(double x);

int main()
{
    square(10);     // ❌ AMBIGUITY!
    // int → long   (ممكن للدالة الأولى)
    // int → double (ممكن للدالة الثانية)
    // المُصرِّف يرفض التجميع كلياً

    square(10L);    // ✅ Exact Match → الدالة الأولى
    square(10.0);   // ✅ Exact Match → الدالة الثانية
}
```

[منطقة الفخ #2 — فخ نوع الإرجاع]:

الطالب يُضيف دالة مُحمَّلة زائدة تختلف فقط في نوع الإرجاع ظناً أنها مقبولة — النتيجة **Compile Error** لأن نوع الإرجاع **لا يُعدّ** معياراً للتمييز

4 جدول الحالات القانونية وغير القانونية

التعريف	الحالة	السبب
`int add(int a, int b);` `double add(double a, double b);`	✅ Legal	يختلفان في نوع الوسائط
`int vol(int s);` `double vol(double r, double h);`	✅ Legal	يختلفان في عدد الوسائط
`int sum(int a, int b);` `float sum(int c, int d);`	❌ Illegal	نفس المعاملات، فرق في الإرجاع فقط

`void f(int x, double y);`
`void f(double x, int y);`
استدعاء: `f(1, 2)`
	❌ Ambiguous	كلتاهما تحتاج تحويلاً للـ `int` الثاني

`long sq(long n);`
`double sq(double x);`
استدعاء: `sq(5)`
	❌ Ambiguous	بنفس المستوى `double` أو `long` يتحول لـ `int`

5 بنك 3 — Tracing أسئلة امتحانية

🔴 تمرين Tracing #1 — حدد السطر الخطأ

cpp

```cpp
int add(int a, int b)
{ return a + b; }

float add(int a, int b)          // سطر A
{ return (float)(a + b); }

double add(double a, double b) // سطر B
{ return a + b; }

int main()
{
    cout << add(2, 3);           // سطر C
    cout << add(2.0, 3.0);       // سطر D
    cout << add(2, 3.0);         // سطر E
    return 0;
}
```

السطر	النتيجة	السبب
سطر A	❌ Compile Error	تكرار نفس المعاملات مع إرجاع مختلف فقط
سطر C	✅ تُنفَّذ الأولى	`int, int` مع Exact Match
سطر D	✅ تُنفَّذ الثالثة	`double, double` مع Exact Match
سطر E	❌ Ambiguity	`int`؟ أم `3.0` لـ`double` يتحول لـ `2`

🟡 تمرين Tracing #2 — What is the output?

```cpp
void show(int x)
{ cout << "INT: " << x << endl; }

void show(double x)
{ cout << "DOUBLE: " << x << endl; }

int main()
{
    show(10);
    show(3.14);
    show('A');
    return 0;
}
```

الاستدعاء	الخطوة المُطبّقة	المخرج
`show(10)`	Exact Match	`INT: 10` ✅
`show(3.14)`	Exact Match	`DOUBLE: 3.14` ✅
`show('A')`	Promotion (`char` → `int`)	`INT: 65` ✅

🟢 تمرين Tracing #3 — What is the output?

```cpp
int volume(int s)
{ return s * s * s; }

double volume(double r, double h)
{ return 3.14 * r * r * h; }

int main()
{
    cout << volume(3) << endl;
    cout << volume(2.5, 5.0) << endl;
    cout << volume(4, 5) << endl;
    return 0;
}
```

الاستدعاء	القيم	المخرج
`volume(3)`	`s=3`	27 ✅
`volume(2.5, 5.0)`	`r=2.5, h=5.0`	98.125 ✅
`volume(4, 5)`	كلاهما `int`	❌ Ambiguity Error — يتحول لـ `double` أو يبقى `int`؟

🏆 سؤال MCQ متوقع — من التنافسي

[سؤال امتحاني #1 — الأكثر تكراراً]:

> أيٌّ مما يلي لا يُستخدم معياراً للتمييز بين الدوال الزائدة؟
>
> - أ) عدد الوسائط
> - ب) نوع الوسائط
> - ج) ✅ نوع الإرجاع (Return Type)
> - د) ترتيب الوسائط